Asset Library

Cross-project asset sharing for Unreal Engine

Version: 2.6.2 | **Engine versions:** UE 5.1 – 5.7 | **Platform:** Win64 (Editor)

This document is the public reference for the **Asset Library** plugin distributed through Fab. It covers what the plugin does, installation, every feature, supported engine versions, known limitations and how to report bugs.

1. What it does

Asset Library is an editor-only Unreal Engine plugin that gives you a persistent, cross-project shelf for the assets you want to reuse — characters, materials, blueprints, levels, animations, anything in the Content Browser. Add an asset to the library from any project, open the library from any other project, and import the asset back with one click. Every hard dependency goes with it, Migrate-style.

The library is just a folder on disk, configurable in Project Settings. Point it at a local folder, a network share or a cloud-synced directory (Dropbox, OneDrive, Google Drive) and the same library works across every project you open.

2. Install

Two delivery formats

The plugin ships in two forms. Pick the one that matches what you downloaded:

- **Pre-compiled binary** (the Fab marketplace builds): drop-in. The archive's `AssetLibrary/` folder contains a `Binaries/Win64/` subfolder with the compiled `.dll`. Works in **any** Unreal project, including Blueprint-only projects, without a C++ toolchain installed on the user's machine.
- **Source distribution** (the GitHub or direct-share ZIPs): no `Binaries/` folder yet. The editor compiles the plugin the first time the project opens. This **requires the host project to be a C++ project** — Unreal will not invoke the compiler from a Blueprint-only `.uproject`.

Install — pre-compiled binary

- Close the Unreal Editor.
- Unzip the build. The archive contains an **AssetLibrary/** folder, and inside it a `Binaries/` subfolder. If `Binaries/` is missing, you have the source distribution — see the next section.
- Drop the **AssetLibrary/** folder into your project's **Plugins/** directory. Create the directory if it doesn't exist. Final path: `<YourProject>/Plugins/AssetLibrary/AssetLibrary.uplugin`.
- Open your **.uproject**. The editor loads with the plugin enabled — no compilation step. Works in Blueprint-only and C++ projects.

Install — source distribution

- The source distribution must be compiled at least once before it can be used. The first compilation only works inside a **C++ project**. After it has been compiled once for a given engine version, the resulting `Binaries/` folder can be copied into any Blueprint-only project on the same engine version.
- **Step 1 — Compile once in a C++ project.** Create or open a C++ project on the target engine version. Drop the AssetLibrary folder into `<Project>/Plugins/`. Open the `.uproject`. Unreal will prompt to build the plugin — click Yes. After the editor finishes loading you'll find `Plugins/AssetLibrary/Binaries/Win64/` filled with the compiled DLL.
- **Step 2 — Reuse in Blueprint-only projects.** Copy the entire `AssetLibrary/` folder (now including `Binaries/`) into any other project, Blueprint-only or C++. The editor loads it without recompiling.
- If you cannot create a C++ project on the target engine version (VS toolchain mismatch on UE 5.5+, etc.), use a machine where the toolchain works to produce the binary, then copy it.

Why this distinction matters. A Blueprint-only project's `.uproject` does not declare any C++ modules, so Unreal never invokes the build system for it. Plugins that need compilation (anything that ships source instead of pre-built binaries) therefore appear to silently fail in Blueprint-only projects. They aren't broken — Unreal just isn't compiling them.

3. Features

3.1 One-click add from the Content Browser

Right-click any asset in the Content Browser and choose **Add to Asset Library**. The plugin captures the asset and every one of its hard dependencies and copies them into the library folder. Multi-selection works. Whole folder drag-and-drop from the Content Browser onto the library panel works too; the folder's relative structure is mirrored inside the library.

3.2 Library browser that looks like the Content Browser

The Asset Library panel renders tiles with the same thumbnails the Content Browser draws, the same colored type stripe under each tile (skeletal mesh purple, level orange, texture red, material green, blueprint blue, etc.), and the same right-click context flow. Long asset names wrap to multiple lines instead of being truncated.

3.3 Folder tree, search, and class filter

Library-side folders are independent from the source-project folders. Create, rename, move, delete. The toolbar has a free-text search (matches name / class / path) and a class-type filter combo box that narrows the tile view to one type at a time. Combine them to drill into massive libraries instantly.

3.4 Engine-version compatibility tagging

Every asset is stamped at add-time with the engine version that imported it. A small badge in the corner of each tile shows that version (e.g. **`5.4+`**). The properties panel glows **GREEN** when the asset is loadable in the current editor and **RED** when it is not. When you try to import an asset stamped with a newer engine than the project you're in, the plugin pops a confirmation dialog listing the offending assets and lets you cancel.

3.5 Dependency picker

Choose dependencies… opens a modal dialog with the asset's dependencies grouped by class — Materials, Textures, Skeletons, Static Meshes, etc. Each group has a master checkbox that toggles all of its items at once plus individual checkboxes per item. Large groups (more than six entries) start collapsed so the dialog never becomes a wall of text. The root asset is always required and locked. A live counter at the bottom shows *Selected: N of M dependencies*.

3.6 Properties inspector

Selecting a single asset surfaces a properties panel on the right side of the library: class, size, dependency count, folder, *added at*, *last update* (modification time of the `.uasset` in the library), source project name with an indicator telling you whether the original project still exists on disk or has been moved/deleted, and the engine compatibility line.

3.7 Configurable library root

Default storage path: `<UserDir>/UEAssetLibrary` (typically `Documents/UEAssetLibrary` on Windows). Change it under **Edit** → **Project Settings** → **Asset Library**. Point it at a network share to use the library on a team, or at a cloud-synced folder (Dropbox / OneDrive / Google Drive) to keep one library across multiple machines.

4. Engine-version support

The plugin's source code compiles cleanly on Unreal Engine 5.1 through 5.7. Each pre-compiled binary on Fab is built against one specific engine version — use the build that matches the engine your project targets.

Engine	Source	Binary build	Notes
UE 5.1	✓	Yes	Lowest supported version. Tested.
UE 5.2	✓	Yes	Tested.
UE 5.3	✓	Yes	Tested.
UE 5.4	✓	Yes	Tested.
UE 5.5	✓	Yes (in next release)	Build pending verification.
UE 5.6	✓	Yes (in next release)	Build pending verification.
UE 5.7	✓	Yes	Latest. Tested.

Asset format rule: a `.uasset` can be loaded only by an engine version equal to or newer than the one that saved it. The plugin tracks each asset's engine version and warns you before importing one that is too new for your project — see **3.4 Engine-version compatibility tagging**.

5. Storage location and settings

All settings live under **Edit** → **Project Settings** → **Asset Library**.

Setting	Default	What it does
Library Root	<UserDir>/UEAssetLibrary	Absolute path on disk where the library is stored. Set this to a network or c
Include Soft Dependencies	Off	When true, soft references are also captured at add-time. Adds more files t
Overwrite On Import	On	When true, importing replaces files already present in the target project. W
Thumbnail Size	128 px	Tile thumbnail edge length in the library panel. Clamps to 48–256 px.

Manifest file. The library folder also contains a `manifest.json` file that records every added asset plus its metadata (folder, dependencies, source project, engine version, added/updated timestamps). Manifest writes are atomic (tmp + rename) so a crash mid-write does not corrupt it. Old manifest schemas are migrated forward automatically.

6. Known limitations

- **Plugin-dependent assets.** If an asset references content from a plugin that is not enabled in the destination project, the import will produce broken references. Enable the same plugins in both projects.
- **Engine-version one-way upgrade.** A `.uasset` saved by a newer engine cannot be loaded by an older engine. The plugin warns you before importing — but it cannot magically downgrade an asset.
- **Single-writer assumption.** The library is not a multi-user version control system. If two editor instances write to the same library at the same time, the last writer wins. For teams, put the library on a per-user folder or coordinate access through a version-controlled shared drive.
- **Engine content excluded.** Files under `/Engine/...` are never copied into the library. Engine content is assumed to be present in every destination project.
- **Editor-only.** The plugin module is type *Editor*. It has no runtime cost and is not included in packaged games.
- **Platform.** Currently distributed as a Win64 editor build. Mac and Linux are not officially supported.

7. Reporting bugs and feature requests

When filing a bug report please include:

- Engine version (Help → About).
- Plugin version (from `AssetLibrary.uplugin`).
- Steps to reproduce, in order.
- The relevant chunk of the editor log at `<Project>/Saved/Logs/<Project>.log`, filtered for the `LogAssetLibrary` category.
- A screenshot if the issue is visual.

Contact: through the Fab listing's contact link. Response usually within 48 hours.

8. Changelog (recent)

2.6.2 — Fab compliance pass

- Reorganized plugin folders into Fab-expected structure (`Config/`, `Docs/`, `Source/`).
- Added `Config/FilterPlugin.ini` declaring `/Docs/...` as a shippable folder.
- Added `PlatformAllowList` in the `.uplugin` module declaration.

2.6.1 — Side properties panel + visible dep picker

- Properties panel moved to a third splitter column on the right.

- Engine compatibility now displays as **`5.X+`** with green / red glow based on current editor.
- **Choose dependencies…** button added directly to the properties panel.

2.6.0 — Properties panel + dependency picker

- Modal dependency picker grouped by class type, with master + per-item checkboxes.
- Properties inspector showing class, size, dep count, folder, timestamps, source project.
- Manifest schema v4: per-dep class lookup + source-project path stored at add time.

2.5.0 — Engine-version compatibility tagging

- Per-asset engine-version stamp, badge on tile, green / red coloring, import confirmation dialog.
- Manifest schema v3.

2.4.0 — Cross-version hardening

- Version guards added for APIs that changed signature between UE 5.1 and 5.7.
- Content Browser toolbar registration tries a list of candidate menu names.
- Startup log banner reports which version-specific paths are active.

2.3.x — UI polish

- Asset type colored stripe under each tile (matches Content Browser).
- Class type filter combo in the toolbar.
- Long asset names wrap to multiple lines (no more ellipsis).

2.2.x — Thumbnail quality + folder drag

- Folder drag-and-drop with mirrored sub-structure.
- Thumbnail rendering pre-warms shaders before capture.
- Batch operations show progress + cancel.

2.1.x — Engine 5.1 support + manifest schema

- Compiles cleanly on UE 5.1.
- Atomic manifest writes (tmp + rename).

9. License

Distribution through the Fab marketplace is covered by the **Fab License Agreement**. No separate or custom license applies to the plugin source or binaries acquired through Fab.